

SYSTEM

INTERNALS

- What is RAG & Why
- Architecture Deep-Dive
- Design Decisions
- LangGraph Workflow
- Conditional Routing
- HITL Implementation
- Challenges & Trade-offs
- Testing Strategy
- Future Enhancements

Technical

Reference Guide

RAG + LangGraph + HITL

Customer Support Assistant

Document Type: Technical Architecture Reference
Version: 1.0
Sections: 9 Detailed Sections

1 Introduction

What is RAG?

Retrieval-Augmented Generation: an LLM first retrieves relevant knowledge from an external source, then uses that context to produce a grounded answer instead of relying solely on parametric memory.

Why It Is Needed

For customer support systems, raw LLM responses are risky because they may invent policies, unsupported product claims, or steps that do not exist in official documentation. RAG reduces that risk by grounding answers in a known knowledge base.

- Grounds answers in a known, inspectable knowledge base.
- Improves factual consistency across repeated queries.
- Allows content updates through document ingestion, not model retraining.
- Enables confidence-based escalation when the system is uncertain.

Use Case Overview

This project is a customer support assistant for a PDF knowledge base. It answers user questions through a CLI interface (for testing and operations) and a Streamlit chat interface (for interactive usage). When the system cannot answer confidently, it escalates to a human agent via a persistent HITL ticket queue.

2 System Architecture Explanation

Two-Phase Design

The system operates in two distinct phases: **ingestion** (offline, run once) and **runtime inference** (online, run per query). These phases are decoupled, enabling the vector store to be rebuilt without touching the query pipeline.

Phase	Steps	Key Components
Ingestion	Load PDF → Normalize → Chunk → Embed → Store	PyPDFLoader, RecursiveCharTextSplitter, all-MiniLM-L6-v2, ChromaDB
Runtime	Query → Normalize → Retrieve → Prompt → Generate → Route → Respond	LangGraph, ChromaDB Retriever, Groq LLaMA 3.3 70B, Streamlit HITL

Component Interactions

Module	Responsibility
main.py / streamlit_app.py	Entry points — CLI and web UI.
service.py	Coordinates ingestion and request execution.
ingestion.py	PDF loading, text normalization, chunk creation.
vector_store.py	Builds embedding model and persists to Chroma.
retriever.py	Executes similarity search and ambiguity heuristics.
llm_service.py	Builds prompt and invokes Groq LLM.
workflow.py	Defines LangGraph state machine and routing edges.
hitl_store.py	Persists escalation tickets and agent responses.

3

Design Decisions

 Chunk Size Choice

Default: `chunk_size=1000, overlap=200`.

Why reasonable: Large enough to preserve procedural support context. Overlap reduces quality loss when details split across boundaries.

Trade-off: Larger chunks improve coherence but dilute retrieval precision. Smaller chunks sharpen retrieval but lose surrounding context.

 Embedding Strategy

all-MiniLM-L6-v2 via HuggingFaceEmbeddings.

Why chosen: Lightweight, widely used for semantic retrieval, fast for local ingestion.

Configuration: Embeddings normalized to align with cosine similarity in Chroma.

 Retrieval Approach

Top-k similarity search with relevance scores.

Confidence metric: Maximum retrieved similarity score used as routing signal.

Limitation: Less robust than multi-score aggregation or reranking, but sufficient for current scale.

 Prompt Design

Strict grounded prompt — answers only from retrieved context.

Instructions: Answer only from context. Avoid guessing. Stay concise and actionable.

Rationale: Customer support domain favours safe escalation over speculative answers.

4 Workflow Explanation

LangGraph Usage

LangGraph models the runtime flow as a stateful directed graph rather than a single sequential chain. This matters because the system has branching routing behaviour: some questions terminate with an automated answer, while others route to escalation.

Node Responsibilities

Node	Responsibility
<code>input_processing_node</code>	Validates and normalizes the user query. Sets early escalation for invalid input.
<code>retrieve_and_answer_node</code>	Retrieves top-k documents, builds LLM prompt from context, generates grounded answer, captures LLM failures.
<code>decision_node</code>	Inspects retrieval quality, evaluates ambiguity and errors, decides output vs escalation path.
<code>output_node</code>	Returns the final answer for direct-response cases.
<code>escalation_node</code>	Converts escalation reason into a user-facing handoff message.

State Object Fields

Field	Type	Description
<code>query</code>	<code>str</code>	Raw user query from UI or CLI.
<code>cleaned_query</code>	<code>str</code>	Trimmed and normalized query.
<code>retrieved_documents</code>	<code>List[Document]</code>	Top-k chunks from Chroma.
<code>similarity_scores</code>	<code>List[float]</code>	Relevance scores per retrieved chunk.
<code>answer</code>	<code>str</code>	LLM-generated or escalation response text.
<code>confidence</code>	<code>float</code>	Maximum similarity score used as routing signal.
<code>needs_escalation</code>	<code>bool</code>	Whether the query should be handed to a human.
<code>escalation_reason</code>	<code>str</code>	Reason string explaining the escalation.
<code>error</code>	<code>str</code>	Captured runtime error text if LLM fails.

5 Conditional Logic

Routing Decisions

The decision node implements a priority-ordered set of escalation checks. If any condition is true, the query is escalated. Otherwise, the automated answer is returned directly.

■ Direct Answer	Query valid + documents retrieved + confidence $\geq$ 0.30 + not ambiguous + no error.
■ Invalid Input	Query is empty or whitespace only.
■ Missing Context	Retrieval returns zero documents.
■ Low Confidence	Maximum similarity score < SIMILARITY_THRESHOLD (default: 0.30).
■ Ambiguous Query	Query < 4 tokens OR matches heuristic list: 'help', 'issue', 'problem', 'not working', 'what should i do', 'can you assist'.
■ Runtime Failure	LLM invocation throws an exception.

6 HITL Implementation



Human-in-the-Loop

HITL is the safety fallback. When the system cannot confidently answer from retrieved content, it stops and hands off the case rather than improvise. Essential for billing, account problems, policy edge cases, or missing

Escalation Flow

Step	Actor/Component	Action
1	LangGraph	Sets needs_escalation=True and records escalation_reason in state.
2	escalation_node	Converts reason to user-facing message and returns it.
3	Streamlit App	Creates a persistent ticket in data/hitl_tickets.json.
4	Ticket Schema	Stores: ticket_id, session_id, query, reason, confidence, context, status.
5	Agent View	Lists all pending tickets with full context for human review.
6	Agent Response	Agent submits reply; ticket status changes to 'resolved'.
7	Customer Session	Polls store and injects human response into original chat history.

Limitations

- Ticket storage is backed by a JSON file — not suitable for concurrent deployments.
- Application-level HITL loop does not resume LangGraph execution itself.
- No authentication on the Agent View — any user can resolve tickets.

7

Challenges & Trade-offs**Retrieval Accuracy vs Speed**

Higher top_k improves context coverage but increases prompt size and latency. Lower top_k is faster but may miss relevant support details.

**Chunk Size vs Context Quality**

Large chunks preserve complete procedures but reduce retrieval precision. Small chunks increase specificity but risk fragmenting multi-step instructions.

**Cost vs Performance**

Groq-hosted inference gives good interactivity without local model hosting, but introduces dependency on network availability and provider limits.

**Simplicity vs Capability**

Single document source, one retriever strategy, one routing threshold, and heuristic ambiguity detection. Keeps implementation lean but limits production readiness.

8 Testing Strategy

Sample Queries & Expected Outcomes

Query	Expected Outcome	Path
How do I reset my password?	Grounded support answer from knowledge base.	Direct
I was charged twice, what should I do?	Billing escalation or grounded policy answer.	Direct / HITL
help	Escalation — ambiguous query heuristic triggered.	HITL
What discount can you give me today?	Escalation — unsupported query, low confidence.	HITL
(empty input)	Validation message — empty query trapped.	Error

Recommended Automated Tests

- Unit tests for `normalize_text` and whitespace handling edge cases.
- Unit tests for chunking validation rules and `chunk_index` assignment.
- Tests for `is_query_ambiguous` covering all heuristic keywords.
- Graph tests asserting correct decision branches for each routing condition.
- Mock-based tests for LLM failures and empty retrieval scenarios.
- HITL ticket creation, resolution, and customer delivery integration tests.

9

Future Enhancements



Multi-Document Support

Extend ingestion to multiple PDFs with metadata tagging and collection-level filtering. Most natural next step — design is already modular.



Feedback Loop

Capture answer helpfulness signals and use them to tune thresholding, chunking, or retrieval policies.



Memory Integration

Session memory for follow-up questions, keeping support facts grounded in the vector store rather than conversational memory alone.



Production Deployment

API-backed service with persistent graph instances, containerised deployment, monitoring, structured logs, and database-backed HITL ticketing.



Quality Improvements

Add reranking, intent classification, better confidence calibration, source citation formatting, hybrid retrieval, and a real support backend for HITL.